

Control Flow, Functions and the Stack

Introduction to ARM Cortex-M Microprocessors – Chapter 7

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

1	Why This Matters	
2	Flags and Conditions	
3	Branches and Control Structures	
4	The Stack	
5	Subroutines	
6	Nested Functions	
7	The AAPCS	

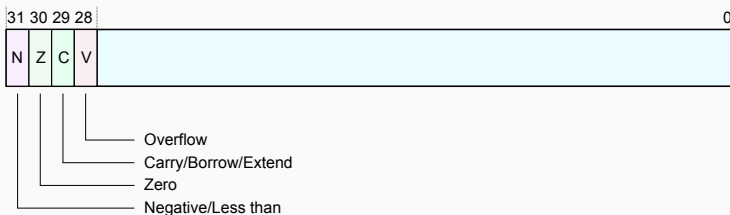
Every `if`, `while` and function call becomes the same two things: a comparison that sets flags, and a branch that tests them. This explains why a signed and an unsigned comparison can differ on the same bits, and why a recursive function can crash a microcontroller. It also prepares you for interrupts, which are function calls made by hardware.

After this chapter, you can:

1. Explain the S suffix and condition codes.
2. Branch on a condition with `CMP`, `TST` and suffixes.
3. Choose signed or unsigned condition codes correctly.
4. Translate C `if/else` and loops to assembly.
5. Trace SP through `PUSH` and `POP` on a full descending stack.
6. Call with `BL`, return with `BX LR`, and explain why nested calls save LR.
7. State the AAPCS rules.

- 1 Why This Matters
- 2 **Flags and Conditions**
- 3 Branches and Control Structures
- 4 The Stack
- 5 Subroutines
- 6 Nested Functions
- 7 The AAPCS

Processor Status Register



Flag	Description
N	1 means result is negative
Z	1 means result is zero
C	unsigned add: 1 means result had a carry out; unsigned subtract: 0 means it had a borrow; shifts: the bit shifted out
V	signed add/subtract: 1 means the result overflowed

The S suffix

```
ADD R0, R1, R2 ; flags unchanged
```

```
ADD S R0, R1, R2 ; updates N Z C V
```

Compare and test — no result kept

Instruction	Computes
CMP Rn, Op2	$Rn - Op2$
CMN Rn, Op2	$Rn + Op2$
TST Rn, Op2	$Rn \& Op2$
TEQ Rn, Op2	$Rn \wedge Op2$

Why compilers care about S

A compiler writes SUBS instead of SUB when a branch right after it needs the flags. So seeing S in a listing tells you **a decision is coming**.

CMP is SUBS that throws the answer away.

Code	Meaning	Flags
EQ	Equal	Z=1
NE	Not equal	Z=0
MI	Negative	N=1
PL	Positive or zero	N=0
VS	Overflow	V=1
AL	Always (default)	—

Code	Meaning	Flags
HS/CS	Unsigned $\geq$	C=1
LO/CC	Unsigned $<$	C=0
HI	Unsigned $>$	C=1, Z=0
LS	Unsigned $\leq$	C=0 or Z=1
GE	Signed $\geq$	N=V
LT	Signed $<$	N $\neq$ V
GT	Signed $>$	Z=0, N=V
LE	Signed $\leq$	Z=1 or N $\neq$ V

Signed and unsigned comparisons are different instructions

GT and HI both mean “greater than.” But one reads N and V, and the other reads C and Z. **Use a signed code for signed data:** for example, $-1 > 5$ is false when signed, but true when unsigned.

- 1 Why This Matters
- 2 Flags and Conditions
- 3 **Branches and Control Structures**
- 4 The Stack
- 5 Subroutines
- 6 Nested Functions
- 7 The AAPCS

A branch writes a new value into the Program Counter. This redirects execution. Four forms matter:

Instruction	Meaning
B label	Unconditional branch – the basic jump
BL label	<i>Branch with link</i> – LR = return address, then jump: a subroutine call
BX Rn	<i>Branch and exchange</i> to an address in a register; BX LR returns
BLX Rn	Call a subroutine whose address is in a register (function pointers)

Function addresses have their low bit set

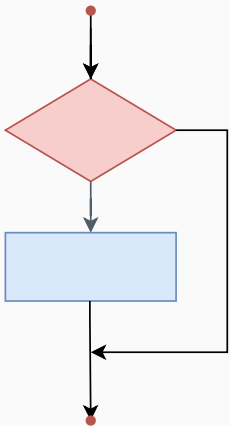
The Cortex-M4 executes only Thumb instructions. **Function addresses therefore have bit 0 set** to mark a valid Thumb target, and BX/BLX check it. In a debugger, this looks like an off-by-one error, until you know to expect it.

Every high-level control structure you are about to see is built from exactly the two pieces just introduced: a **comparison that sets flags**, and a **conditional branch** that tests them. Nothing new is added — only combined.

C construct	Built from
if	one conditional branch
if/else	one conditional branch + one unconditional branch
while	one conditional branch + one backward branch
do/while	one backward conditional branch, no forward branch at all

Keep this map in view through the next few slides: each worked example below is one more arrangement of the same two pieces, never a new mechanism.

The if Construct



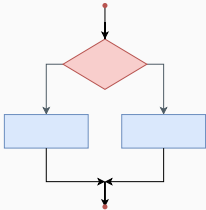
```

x = 10;
if ( x > 0 ) {
    x = x - 1;
}
  
```

```

MOV    r0, #10
CMP     r0, #0
BLE     endif
SUB     r0, r0, #1
endif:
  
```

Read the branch as the opposite of the C condition: the code tests $x > 0$. The assembly branches away with BLE to skip the body when this is not true.



```
if (a == b)
    x = 1;
else
    x = 2;
```

```
P    R0, R1          ; Compare a (R0) and b
                    (R1)
    BNE    else_block ; If Z == 0
                    (not equal), branch to else

; If-block: a == b
MOV    R2, #1        ; x = 1
B      endif         ; Branch
                    past the else block

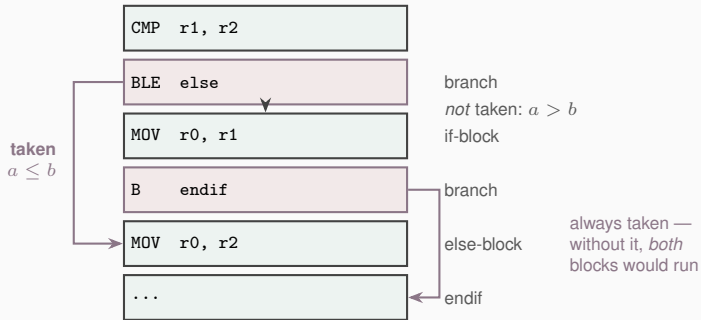
else_block
; Else-block: a != b
MOV    R2, #2        ; x = 2

endif
```

Both arms fall through without B endif

The true arm must jump past the else arm. Without B endif, execution falls through and runs both arms.

The Two Paths Through an if-else

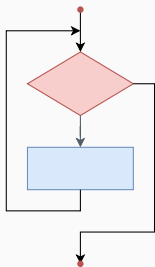


The conditional branch tests the *opposite* condition and jumps over the if-block; the unconditional branch jumps over the else-block. Execution reaches exactly one MOV — never zero, never both.

The while Construct

- Implements both the **while** and **for** keywords in C
- The condition is tested **before** the loop starts: a pre-test loop

sum = 1+2+3+4+5



```
sum = 0;
for (i = 1; i <= 5; i++)
    sum = sum + i;
```

```

MOV    r0, #0
MOV    r1, #1
loop:
    CMP    r0, #5
    BGT    done
    ADD    r0, r0, r1
    ADD    r1, r1, #1
    B      loop
done:
```

Worked Example: Popcount

- **Popcount** (population count) means counting the number of bits that are 1

This example is primarily about **reusing flags generated by another instruction**: LSRS shifts *and* sets flags, so one instruction does the work while needed anyway.

```
int x = 0x5555;    // input
int c = 0;
for (; x != 0; x >>= 1)
    if (x & 1)
        c++;
```

```

loop    LDR    r1, =0x5555
        MOV    r0, #0        ; count = 0

        AND    r2, r1, #1    ; check bit 0
        ADD    r0, r2        ; c++
        LSRS   r1, r1, #1    ; next bit
        BNE    loop

done
```

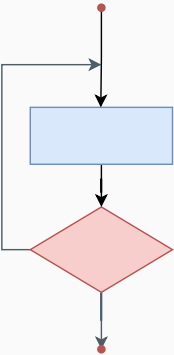
A new principle: one instruction, two jobs

LSRS shifts *and* sets **Z** when the remaining bits reach zero, so the shift itself supplies the condition the next branch needs — no separate comparison. This is the same S-suffix idea from two slides ago, now doing useful work *and* preparing the next decision. Note: if x starts at 0, this loop still runs once.

The do-while Construct

The condition is tested **after** the loop body runs: a post-test loop.

$$\text{sum} = 1+2+3+4+5$$



```

sum = 0;
i = 1;
do {
    sum = sum + i;
    i++;
} while (i <= 5);
  
```

```

MOV    r0, #0
MOV    r1, #1

loop
ADD     r0, r0, r1
ADD     r1, r1, #1
CMP     r1, #5
BLE     loop
  
```

A do-while always runs its body at least once. The comparison comes after the work, not before.

Loops: Counting Down Is Cheaper

The flags are not only for comparisons — an arithmetic instruction with the S suffix can supply them too, exactly as LSRS did in **Popcount**.

Counting up

```

MOV    r0, #0
loop
    ...           ; body
    ADD    r0, #1
    CMP    r0, #10
    BLT    loop
  
```

Three instructions of overhead per iteration.

Counting down

```

MOV    r0, #10
loop
    ...           ; body
    SUBS   r0, #1  ; and set flags
    BNE    loop
  
```

Two — the CMP is gone.

Why the S suffix saves an instruction

SUBS decrements *and* sets **Z** when the result reaches zero, so no separate comparison is needed. This is the most common hand-optimisation in ARM assembly, and compilers do it automatically whenever the loop order does not matter.

Worked Example: Find Maximum

- Find the biggest signed word value in a memory block, starting at a known address

```
int max = 0;
int *ptr = (int *)0x20001000;
int nwords = 64;
do {
    if (*ptr > max)
        max = *ptr;
    ptr++;
    nwords--;
} while (nwords);
```

```

MOV    r0, #0           ; max
LDR    r1, =0x20001000  ; ptr
MOV    r2, #64          ; nwords

loop
    LDR    r3, [r1], #4    ; *ptr, ptr++
    CMP    r0, r3
    BGE    skip
    MOV    r0, r3          ; new max
skip
    SUBS   r2, #1
    BNE    loop
```

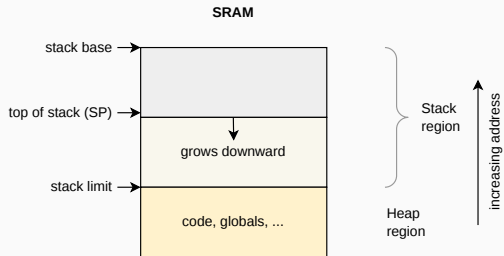
LDR Rn, =value loads a number, not a variable

`ptr` and `nwords` hold a *fixed* starting address and count, known at compile time. The assembly loads those known values directly into `r1/r2` — there is no need to read `ptr/nwords` back out of memory, since nothing else in the program ever changes them.

- 1 Why This Matters
- 2 Flags and Conditions
- 3 Branches and Control Structures
- 4 The Stack**
- 5 Subroutines
- 6 Nested Functions
- 7 The AAPCS

What Is a Stack?

- A **stack** is a region of ordinary SRAM managed using the **Stack Pointer (SP, R13)**.
- It follows a **last-in, first-out (LIFO)** rule: the most recently stored item is the first one removed.
- The stack can be located anywhere in the memory map.
- It is commonly used to hold **return addresses**, **saved registers**, and **local variables**.
- All subroutines typically share the same stack.



- The stack is typically initialized at the **top of SRAM**.
- Initially, SP points to the top of the stack region.
- As items are pushed, SP moves toward **lower addresses**.
- The heap, when present, typically grows upward from the opposite direction.

How Does the Stack Grow?

- The stack grows and shrinks by moving the **Stack Pointer (SP)**.
- On a 32-bit Cortex-M processor, each stack word occupies **4 bytes**.

PUSH

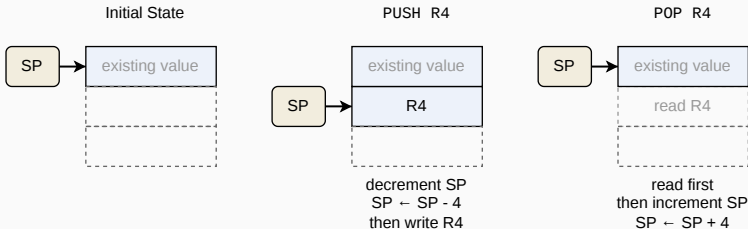
1. Move SP down by 4 bytes:

$$SP \leftarrow SP - 4$$
2. Store the value at the new SP address.

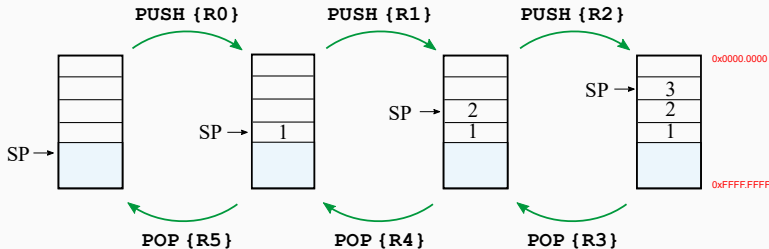
POP

1. Read the value at the current SP address.

$$SP \leftarrow SP + 4$$
2. Move SP up by 4 bytes:



Why Is It Called Last-In, First-Out?



Three PUSH operations place three registers on the stack. Because each new value is placed on top of the previous one, the first POP retrieves the value pushed last.

The LIFO rule

The **last value pushed** is the **first value popped**.

- A subroutine may use the stack temporarily to save registers or other data.
- Before returning, it must restore the stack to the state it had on entry.
- In simple terms:

Every PUSH must eventually have a matching POP.

Example

3 registers pushed $\longrightarrow$ SP moves down by $3 \times 4 = 12$ bytes $\longrightarrow$ 3 registers popped $\longrightarrow$ SP returns to its original value

The order matters too: values must be popped in the **reverse order** in which they were pushed.

The stack is ordinary memory. If SP moves outside the space or contents intended for the stack, the program can behave unpredictably.

Too many PUSHes

- SP keeps moving toward lower addresses.
- The stack can exceed its allocated region.
- It may collide with other memory regions.
- This is commonly called **stack overflow**.

Too many POPs

- SP moves beyond the valid stack contents.
- A POP may read data that was never pushed.
- The result may be incorrect data or a fault.
- This is commonly called **stack underflow**.

- 1 Why This Matters
- 2 Flags and Conditions
- 3 Branches and Control Structures
- 4 The Stack
- 5 Subroutines**
- 6 Nested Functions
- 7 The AAPCS

Why Subroutines Exist

As programs grow, the same instructions are often needed in more than one place. Rather than copying them, place them in one block and **call** that block whenever it is needed.

```
uint32_t read_gpioport(void) {  
    return GPIOA->IDR;  
}
```

A caller simply writes

```
uint32_t state = read_gpioport();
```

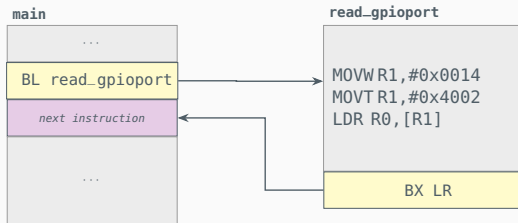
The same idea applies in assembly: the instructions that read the port live in one place, and the program branches there whenever the value is needed — with one new requirement. A subroutine must **return to the instruction after the call**.

This is how assembly actually shows up in real firmware

Almost nobody writes a whole project in assembly. The usual pattern is **C code — main() and everything it calls — with one or two hand-tuned routines written in assembly and linked in**. `read_gpioport` is exactly that: C calls it like any other function, with no idea its body is assembly.

Calling and Returning: BL and BX LR

- BL branches to a subroutine and records where execution should return.
- The target address is loaded into the PC, while the address of the instruction after BL is placed in LR.
- LR therefore holds the return address for the subroutine.
- Rather than pushing the return address onto the stack, ARM initially keeps it in the LR register.



BL calls a subroutine; BX LR returns from it.

Returning a Value

How does a subroutine hand a result back? By another convention: the return value goes in **R0**.

```
read_gpioport
    MOVW    R1, #0x0014    ; GPIOA_IDR low half
    MOVT    R1, #0x4002    ; GPIOA_IDR high half
    LDR     R0, [R1]       ; read GPIO input register
    BX      LR             ; return, with result in R0
```

```
BL    read_gpioport
; R0 now contains the GPIO port value
```

`read_gpioport` takes no arguments, yet still returns a value — the two conventions are independent. The next slide shows `R0` used for an argument **too**.

Passing Arguments

Now consider a function that needs information *from* its caller:

```
uint32_t read_bit(uint32_t port, uint32_t mask) {
    return port & mask;
}
```

The first four 32-bit arguments travel in R0–R3:

- R0 = port
- R1 = mask
- R0 = result, on return

```
read_bit
    AND    R0, R0, R1
    BX     LR
```

R0 therefore plays two roles at different points in the same call: it carries a value **in**, as the first argument, and it carries the result **out**, once the function returns.

R0–R1 for arguments, R0 for the return value — this is not a rule invented for one function. It is a shared convention every function obeys. Two more leaf subroutines show it holding under slightly different conditions, before asking what happens once one subroutine calls another.

Worked Example: A Value That Must Survive

`read_gpioport` and `counter` share the same shape — no parameters, one result in `R0`, a leaf function — but `counter` needs something `read_gpioport` did not: a value still there the **next** time it is called.

```
int counter(void) {
    static int n = 0;
    return ++n;
}
```

```
counter
    LDR    R0, =n
    LDR    R1, [R0]
    ADDS   R1, R1, #1
    STR    R1, [R0]
    MOV    R0, R1
    BX     LR
```

A register cannot hold `n`

Every register's contents are gone the moment the function returns. `n` must live in **memory**, at a fixed address, like any global or static variable. The register traffic here only gets that value into `R0` and back out again.

Worked Example: `x = foo(y, z)`

`read_bit` returned the result of a single AND. A real function body is rarely that short:

```
int foo(int a, int b) {
    return a * 2 + b;
}
```

```
x = foo(y, z);
```

```
foo
```

```
    LSLS    R0, R0, #1    ; a*2
    ADDS    R0, R0, R1    ; +b
    BX      LR
```

```
; y already in R0, z in R1
```

```
    BL      foo
    MOVS    R4, R0        ; x = result
```

Same convention, more work inside

`a*2` is the shift-instead-of-multiply trick from Chapter 6 — the calling convention changes nothing about how the body is optimised. `R0` still carries the argument in and the result out, exactly as `read_bit` showed; only the number of instructions in between has grown.

The Internals Can Change Without the Caller Knowing

Every example so far has had exactly one implementation. Consider a function called exactly like any other:

```
int popcount(uint32_t x);
...
int n = popcount(mask);
```

A straightforward version checks every bit, shifting right until nothing remains — the same loop as the earlier Popcount example, now wrapped as a callable function:

```
popcount
    MOV    R1, #0          ; count = 0
loop
    ANDS   R2, R0, #1      ; check bit 0
    ADD    R1, R1, R2      ; count += bit
    LSRS   R0, R0, #1      ; next bit
    BNE    loop
    MOV    R0, R1
    BX     LR
```

A Faster popcount: Same Signature

One loop iteration per **bit position** — 32 in the worst case. A well-known trick does better: $x \& (x - 1)$ clears the **lowest set bit**, so the loop runs once per 1 bit, not once per position.

```

popcount
    MOV    R1, #0          ; count = 0
    CMP    R0, #0
    BEQ    done
loop
    ADD    R1, R1, #1      ; count++
    SUBS   R2, R0, #1      ; R2 = x - 1
    ANDS   R0, R0, R2      ; x &= (x - 1)
    BNE    loop
done
    MOV    R0, R1
    BX     LR
  
```

Same signature, same calling sequence — different caller cannot tell

Argument in R0, result in R0, BX LR to return — identical to the first version. A function's internals can be rewritten and replaced, and every caller keeps working untouched.

- 1 Why This Matters
- 2 Flags and Conditions
- 3 Branches and Control Structures
- 4 The Stack
- 5 Subroutines
- 6 Nested Functions**
- 7 The AAPCS

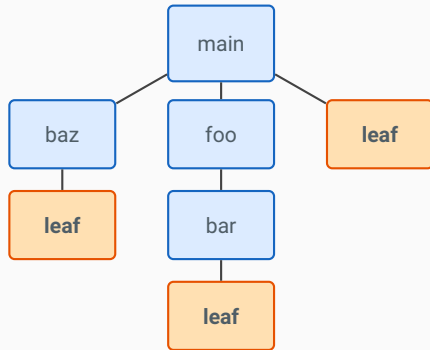
Leaf Versus Non-Leaf Subroutines

- A **leaf** subroutine does not call another subroutine. It executes no BL, so its LR remains unchanged.
- A **non-leaf** subroutine calls another subroutine. Its next BL will overwrite the value currently in LR.

This gives us an important rule:

A non-leaf subroutine must save its LR before making another call.

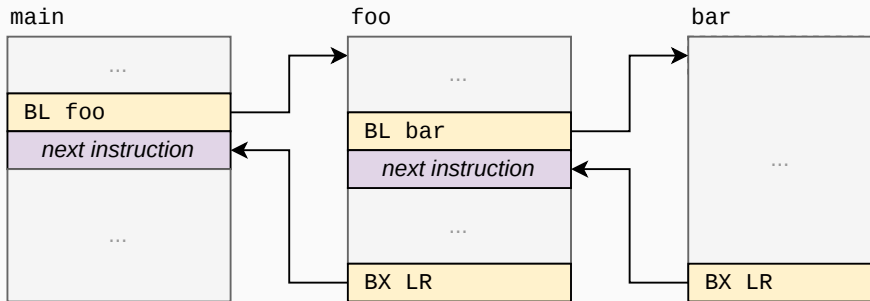
The stack provides a convenient place to save it.



Filled boxes are leaf subroutines. Plain boxes call another subroutine and therefore **must preserve LR**.

The Problem: A Second BL Overwrites LR

Consider this three-level call: main calls foo, and foo calls bar.



When main executes BL foo, the return address for main is placed in LR.

But then foo executes BL bar. This places a new return address in the same register, **overwriting the original address**.

Now bar can return to foo, because LR contains the correct address for that return. However, the original return address back to main has been lost.

BL AddNumbers

AddNumbers

```
PUSH {LR}      ; save caller's return address
ADD  R0, R0, R1
POP  {PC}      ; restore and return
```

Before making another call, a non-leaf subroutine can move its return address from LR to the stack:

```
PUSH {LR}
```

This preserves the address even when a later BL overwrites LR.

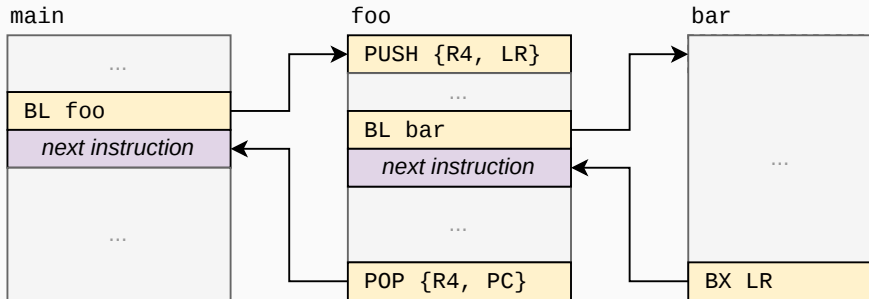
At the end, POP {PC} restores the saved address directly into the program counter, returning execution to the caller.

The key idea

LR is convenient for a single call, but it cannot preserve multiple return addresses.

The stack turns one temporary register into persistent storage for the return address.

A leaf subroutine does not need this step: because it makes no further call, its LR remains intact and it can return with BX LR.



PUSH can save several registers in one instruction. For example, `PUSH {R4, LR}` saves both R4 and the return address on the stack.

When the subroutine finishes, the same registers can be restored with `POP {R4, PC}`.

Restoring LR into PC returns execution to the caller, while restoring R4 returns it to its original value.

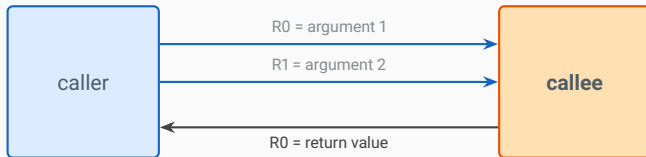
The examples so far passed arguments in R0/R1 and returned a result in R0, by convention. The nested-call problem just solved — save LR before a second BL — raises the same question one level up: **which** registers are safe to overwrite, and which must a callee restore?

Every function you write, and every library function you call, needs the *same* answer to that question — otherwise C code and hand-written assembly could never call each other safely. That shared answer is a formal convention: the **AAPCS**.

- 1 Why This Matters
- 2 Flags and Conditions
- 3 Branches and Control Structures
- 4 The Stack
- 5 Subroutines
- 6 Nested Functions
- 7 The AAPCS**

How Do Two Functions Agree on Anything?

Your C code and a library compiled years ago must agree on where arguments go, where results return, and who preserves what — with no run-time negotiation.



Procedure Call Standard for the ARM Architecture (AAPCS)

The AAPCS is that agreement

It defines where arguments and return values go and which registers a function must preserve. This lets C and assembly functions call each other safely.

Registers	Rule
R0–R3	Arguments 1–4; R0 also carries the return value. Caller-saved : the callee may overwrite them.
R4–R11	Callee-saved : the callee must restore any it uses.
R12	Scratch register.
R13–R15	SP, LR, PC.

Arguments beyond the fourth go on the **stack** — the same mechanism just used to save LR.

Do No Damage: Callee-Saved Registers

A function may use R4–R11, but it must restore them before returning. The pattern is the same as saving LR: **save, use, restore**.

```
foo  PUSH {R4, R5}      ; save registers this function will use
     MOV  R4, #0
     MOV  R5, #0xAB
     ADD  R4, R4, R5
     ...
     POP  {R4, R5}      ; restore their original values
     BX   LR
```

Why this matters

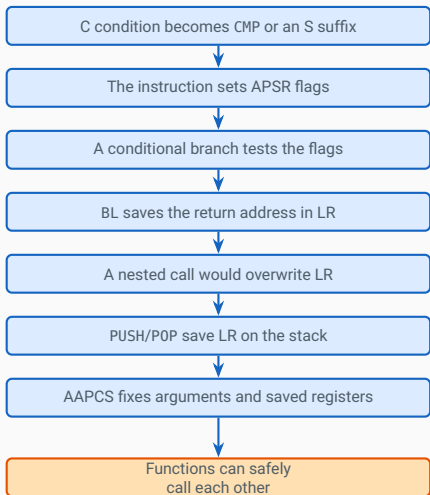
The caller can continue using R4 and R5 after foo returns, knowing their values are unchanged. No special agreement is needed at each call site.

A Function Call Viewed at Five Levels

Level	Representation
Physical hardware	SP drives the address bus; SRAM cells hold the saved LR
Register	R0, R1 carry the arguments; LR holds the return address
C	<code>sum = AddNumbers(5, 10);</code>
Assembly (call)	<code>MOV R0, #5 / MOV R1, #10 / BL AddNumbers</code>
Assembly (callee)	<code>PUSH {LR} ... POP {PC}</code>

One call, five simultaneous descriptions

Chapters 1–7 have all built this table. The C statement reaches a hardware SRAM cell, through registers, instructions, and bytes.



Terminology covered

- APSR flags, condition codes
- Signed vs unsigned suffixes
- BL, BX LR
- Full descending stack
- AAPCS

Next: Chapter 8

General Purpose Input Output

The book stops being theory. A physical pin changes voltage.